■ Design Real-Time Leaderboard

System Design Interview | Counting & Ranking / Async Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Update player score in real-time; Query top-K players globally; Get player's current rank; Time-windowed leaderboards (daily/weekly/all-time)
Non-Functional	Score update <100ms; Rank query <50ms; Handle 10M+ players; Real-time updates pushed to viewers; Accurate ranking at scale
Scale	Gaming apps: millions of score updates/sec during peak; global leaderboards vs per-region; time-decay scoring

■ Architecture Overview

• Score Event → Kafka → Score Processor → Redis Sorted Set (real-time rank) + PostgreSQL (persistent scores)

• Redis Sorted Set: ZADD leaderboard score player_id (O log N); ZREVRANK for player rank (O log N); ZREVRANGE 0 99 for top-100 (O log N + K)

• Time-windowed: separate Redis keys per window — leaderboard:daily:2024-01-15, leaderboard:weekly:2024-W03; expire with TTL

• Push updates: WebSocket connections to leaderboard service; publish rank changes to Pub/Sub → push to watching clients

■ Redis Sorted Set Deep Dive

• **ZADD:** ZADD leaderboard 1500 'player:123' — add/update score; O(log N); if player exists score is replaced

• **ZINCRBY:** ZINCRBY leaderboard 50 'player:123' — atomically increment score by delta; better for additive scoring

• **ZREVRANK:** ZREVRANK leaderboard 'player:123' — returns 0-based rank (0 = top); O(log N); returns nil if not found

• **ZREVRANGE:** ZREVRANGE leaderboard 0 99 WITHSCORES — top 100 with scores; O(log N + K); use for leaderboard page

• **ZRANGEBYSCORE:** ZRANGEBYSCORE leaderboard 1000 +inf — all players above score threshold; useful for tier brackets

■ Score Update Pipeline

• Game server → POST /score {player_id, delta, game_id, timestamp} → Score Service → validate → ZINCRBY in Redis

• Async persistence: score event published to Kafka → consumer writes to PostgreSQL (player_id, total_score, updated_at)

• Score validation: server-side anti-cheat; score delta must be within plausible range per game action; flag outliers

• Batch updates: for high-frequency games (1 update/sec/player), buffer 5s of deltas locally → single ZINCRBY to Redis

■ Sharding for Scale (10M+ Players)

• Single Redis sorted set handles ~10M members comfortably (few GB memory); beyond that → shard by player_id range or hash

• Sharded top-K: each shard computes local top-K → merge and re-rank (scatter-gather); O(shards × K) merge cost

• Regional leaderboards: separate sorted sets per region; global leaderboard = top players from each region merged daily

• Score tiering: segment into Bronze/Silver/Gold tiers (score ranges); each tier has own sorted set; reduces single set size

■■ Rank Query Approaches

Redis ZREVRANK (Real-Time)	Pre-computed Rank Table
✓ Always accurate rank	✓ O(1) read from DB
✓ O(log N) per query	✓ Pre-computed every N seconds
✓ Instant after score update	✓ Stale by up to N seconds
✓ Redis memory scales with players	✓ Better for display-only use

■ Time-Windowed Leaderboards

• Daily: key = leaderboard:daily:{YYYY-MM-DD}; TTL = 25h; ZINCRBY on each score event; expire at midnight + buffer

• Weekly: key = leaderboard:weekly:{YYYY-WNN}; TTL = 8 days; same pattern; top-K query for weekly prizes

• All-time: persistent Redis key + backed by PostgreSQL; Redis is cache, DB is source of truth

• Score reset: for new season, create new Redis key (don't ZREMRANGEBYRANK) — keeps history; old key expires naturally